

TUGAS UJIAN TENGAH SEMESTER

Semester Genap



Mata Kuliah : Pembangunan Perangkat Lunak Berorientasi Service (SE.2)
Dosen Pengampu : Muhammad Panji Muslim, S.Pd., M.Kom
Kelas : A-IF
Nama : Rezaul Karim
NIM : 2310511134

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA
FAKULTAS ILMU KOMPUTER
PRODI S-1 INFORMATIKA

Jl. RS. Fatmawati No. 1, Pondok Labu, Jakarta Selatan 12450

Telepon: (021) 7656971, Fax: (021) 7656904

Laman: www.upnvj.ac.id, Pos-el: upnvj@upnvj.ac.id

Pendahuluan

Sistem Reservasi Klinik ini dikembangkan untuk memfasilitasi proses pendaftaran pasien, pengelolaan jadwal dokter, dan pencatatan reservasi secara terkomputerisasi. Tujuan utamanya adalah mendigitalisasi alur layanan administrasi kesehatan agar menjadi lebih efisien, transparan, dan terstruktur.

Untuk memenuhi tuntutan sistem yang modern, aplikasi ini tidak dibangun menggunakan pendekatan perangkat lunak monolitik tradisional, melainkan dirancang secara khusus menggunakan arsitektur *Service-Oriented*. Pendekatan ini memecah fungsionalitas aplikasi menjadi beberapa layanan independen yang saling berkomunikasi, guna mencapai tingkat skalabilitas dan keandalan sistem yang jauh lebih tinggi.

Arsitektur Microservices

Sistem ini dipecah menjadi tiga layanan utama yang berdiri sendiri, yaitu auth-service, medical-service, dan reservation-service. Pemisahan fungsionalitas ini merupakan keputusan arsitektural yang diwajibkan untuk menghindari kelemahan pendekatan monolitik. Berikut adalah justifikasi utama di balik pemisahan layanan tersebut:

- **Pemisahan Tanggung Jawab**

Pendekatan monolitik menggabungkan seluruh logika dalam satu basis kode yang rawan konflik. Dengan arsitektur ini, setiap layanan memiliki fokus domain spesifik—auth-service murni mengurus keamanan, medical-service menangani data rekam medis, dan reservation-service mengelola alur antrean—sehingga pemeliharaan kode dapat dilakukan secara terisolasi.

- **Skalabilitas Independen**

Pada aplikasi monolitik, lonjakan pengunjung pada satu fitur mengharuskan penambahan kapasitas (*scaling*) pada seluruh server. Melalui pemisahan ini, apabila fitur antrean pasien membludak, sistem hanya perlu meningkatkan sumber daya pada reservation-service tanpa membebani operasional layanan medis.

- **Toleransi Kegagalan**

Kendala pada satu komponen di sistem monolitik berpotensi melumpuhkan seluruh aplikasi. Dalam arsitektur *microservices* ini, apabila reservation-service mengalami *downtime*, staf medis klinik dipastikan tetap dapat mengakses dan memperbarui data pasien melalui medical-service secara stabil dan aman.

API Gateway & Peta Routing

Sebagai arsitektur *microservices*, sistem ini memanfaatkan API Gateway sebagai pintu gerbang tunggal bagi seluruh request dari klien, seperti aplikasi frontend maupun Postman. Selain bertugas mengarahkan rute (*routing*) ke masing-masing service, API Gateway ini juga memikul tanggung jawab krusial untuk melakukan validasi token JWT secara terpusat sebelum meneruskan request, serta menerapkan pembatasan akses sebesar 60 request/menit untuk mencegah spam dan eksploitasi.

Berikut adalah peta *routing* yang diimplementasikan pada API Gateway:

Endpoint Klien (Via API Gateway)	Metode HTTP	Layanan Tujuan (Target Service)
/auth/api/auth/* (<i>Login, Refresh, Logout</i>)	POST	auth-service
/api/medical/patients	GET, POST, PUT, DELETE	medical-service
/api/medical/doctors	GET, POST, PUT, DELETE	medical-service
/api/medical/schedules	GET, POST, PUT, DELETE	medical-service
/api/medical/prescriptions	GET, POST, PUT, DELETE	medical-service
/api/reservation	GET, POST	reservation-service

Database & Implementasi Layanan

Untuk menjaga prinsip desentralisasi *microservices*, setiap layanan di dalam sistem ini diwajibkan memiliki *database* atau skema yang sepenuhnya terpisah dan tidak membagikan tabel secara langsung antar *service*.

- Medical Service (PHP CodeIgniter 4)

Layanan inti medis ini dibangun menggunakan kerangka kerja CodeIgniter 4 dengan menerapkan pola arsitektur MVC (*Model-View-Controller*) secara ketat, di mana logika terpisah dengan jelas.

- Layanan ini menggunakan basis data relasional MySQL (*linik_medical*) yang menaungi 4 tabel terelasi:
 - patients,
 - doctors
 - schedules
 - prescriptions.
- Untuk menjaga integritas, layanan ini mengimplementasikan validasi input pada layer controller serta menyediakan fitur paging (halaman) dan filtering pada endpoint listing data.

- Reservation Service (Node.js)

Layanan ini difokuskan pada alur bisnis penjadwalan pasien dan beroperasi menggunakan basis datanya sendiri, yaitu *reservation_db*. Layanan ini dirancang untuk

tidak berdiri sendiri; alih-alih, ia secara aktif mengonsumsi *endpoint* dari *medical-service* (melakukan komunikasi *inter-service*) guna memvalidasi pembuatan reservasi.

Komunikasi Inter-service

Mengingat penerapan prinsip *Database per Service*, pembuatan sebuah reservasi antrean baru memerlukan sinkronisasi dan pengecekan silang antar aplikasi. Proses ini terjadi melalui komunikasi *inter-service* secara *synchronous* berbasis REST API dengan alur sebagai berikut:

1. Penerimaan Request

Klien mengirimkan instruksi POST ke endpoint `/api/reservation` melalui Gateway, dengan membawa muatan data berupa `patient_id` dan `schedule_id`.

2. Validasi Eksternal

Sebelum mencatat reservasi, *reservation-service* (Node.js) akan bertindak layaknya klien internal dan mengirimkan HTTP GET Request ke *medical-service* (`/api/medical/patients/{id}`) untuk memastikan eksistensi pasien, dan ke `/api/medical/schedules/{id}` untuk memverifikasi ketersediaan jadwal dokter.

3. Eksekusi Transaksi

Apabila kedua request tersebut mengembalikan status HTTP 200 OK (data ditemukan secara valid), barulah *reservation-service* mengeksekusi penyimpanan data reservasi tersebut ke dalam `reservation_db`. Sebaliknya, apabila data tidak ditemukan (status HTTP 404 Not Found), transaksi reservasi akan ditolak secara otomatis.

Otorisasi OAuth 2.0 & Keamanan JWT

Keamanan akses dalam sistem layanan mikro ini diatur melalui implementasi dua lapis otentikasi dan otorisasi yang ketat:

- Google OAuth 2.0

Sistem mendukung *Single Sign-On* (SSO) menggunakan penyedia layanan Google dengan menerapkan alur persetujuan *Authorization Code* (bukan *Implicit Flow* yang sudah usang). Identitas hasil otorisasi (termasuk nama, email, dan foto profil) kemudian dipetakan secara otomatis ke dalam *database* pengguna lokal (`auth_db`) dengan memberikan tanda khusus (*flag*) `oauth_provider`.

- Otentikasi Terpusat Berbasis JWT

Pasca-login, sistem akan menerbitkan *Access Token* JWT berdurasi pendek (maksimal 15 menit) untuk mengakses sumber daya layanan, dan *Refresh Token*

(berdurasi 7 hari) untuk memperbarui sesi tanpa perlu *login* ulang. Sistem juga menyediakan *endpoint* khusus untuk pencabutan token saat pengguna *logout*. Penting ditekankan bahwa validasi keabsahan JWT ini tidak dititipkan pada masing-masing layanan medis atau reservasi, melainkan dieksekusi secara terpusat sebagai "satpam" (*middleware*) di *layer* API Gateway sebelum rute diteruskan ke *service* di belakang layar.